

```
In [1]: import numpy as np
import pandas as pd
import os
```

Import Amman Data

```
In [2]: folder = r"C:\Users\user\OneDrive\Desktop\new ap mort\AP data\Amman/" #Define the directory of folder containing Amman Station
files = [file for file in os.listdir(folder) if file.endswith('.xlsx')]
amm = [pd.read_excel(folder + file) for file in files]
files = [file[2:5] for file in files]
for i in range(len(files)):
    amm[i]['Sta'] = files[i] #add a column containing each station's symbol
```

Import Irbid Data

```
In [3]: folder = r"C:\Users\user\OneDrive\Desktop\new ap mort\AP data\Irbid/" #Define the directory of folder containing Irbid Station
files = [file for file in os.listdir(folder) if file.endswith('.xlsx')]
irb = [pd.read_excel(folder + file) for file in files]
files = [file[2:5] for file in files]
for i in range(len(files)):
    irb[i]['Sta'] = files[i] #add a column containing each station's symbol
```

Import Zarqa Data

```
In [4]: folder = r"C:\Users\user\OneDrive\Desktop\new ap mort\AP data\Zarqaa/" #Define the directory of folder containing Zarqa Station
files = [file for file in os.listdir(folder) if file.endswith('.xlsx')]
zar = [pd.read_excel(folder + file) for file in files]
files = [file[2:5] for file in files]
for i in range(len(files)):
    zar[i]['Sta'] = files[i] #add a column containing each station's symbol
```

Concatenate the data of each city

```
In [5]: amm = pd.concat(amm, ignore_index=True)
irb = pd.concat(irb, ignore_index=True)
zar = pd.concat(zar, ignore_index=True)
```

Define the `imputation` function

```
In [ ]: def impute(df, col):
        """
        Impute missing values in the specified column of the DataFrame using a ratio-based method.

        df: pandas DataFrame containing the data
        col: string, the name of the column to impute

        """
        for day in df['Date'].unique():
            year = pd.to_datetime(day).year
            sdf = df[df['Date'] == day]
            nandf = sdf[sdf[col].isna()]
            if not nandf.empty:
                mean_value = sdf[col].mean()
                if len(nandf) == 1:
                    sta_ann_mean = df[(df['Sta'] == nandf['Sta'].values[0]) & (df['Date'].dt.year == year)][col].mean()
                    rest_ann_mean = df[(df['Sta'] != nandf['Sta'].values[0]) & (df['Date'].dt.year == year)][col].mean()
                    df.loc[nandf.index, col] = mean_value * (sta_ann_mean / rest_ann_mean)
                if len(nandf) > 1:
                    for i in range(len(nandf)):
                        sta_ann_mean = df[(df['Sta'] == nandf['Sta'].values[i]) & (df['Date'].dt.year == year)][col].mean()
                        rest_ann_mean = df[(df['Sta'] != nandf['Sta'].values[i]) & (df['Date'].dt.year == year)][col].mean()
                        df.loc[nandf.index[i], col] = mean_value * (sta_ann_mean / rest_ann_mean)

        return None
```

For PM10, check # of missing values for each city

```
In [17]: print(f"Amman: {len(amm[amm['PM10'].isna()])}")
          print(f"Irbid: {len(irb[irb['PM10'].isna()])}")
          print(f"Zarqa: {len(zar[zar['PM10'].isna()])}")
```

Amman: 480

Irbid: 680

Zarqa: 470

Perform the imputation for PM10

```
In [20]: impute(amm, 'PM10')
         impute(irb, 'PM10')
         impute(zar, 'PM10')
```

Check # of PM10 missing values after imputation

```
In [21]: print(f"Amman: {len(amm[amm['PM10'].isna()])}")
         print(f"Irbid: {len(irb[irb['PM10'].isna()])}")
         print(f"Zarqa: {len(zar[zar['PM10'].isna()])}")
```

Amman: 180

Irbid: 400

Zarqa: 126

Similarly for NO2

```
In [22]: print(len(amm[amm['NO2'].isna()]))
         print(len(irb[irb['NO2'].isna()]))
         print(len(zar[zar['NO2'].isna()]))
```

573

188

140

```
In [23]: impute(amm, 'NO2')
         impute(irb, 'NO2')
         impute(zar, 'NO2')
```

```
In [24]: print(len(amm[amm['NO2'].isna()]))
         print(len(irb[irb['NO2'].isna()]))
         print(len(zar[zar['NO2'].isna()]))
```

192

66

62